

QuizMaster-V2 Project Documentation

Personal Details

Name: Krishneal Dutta

Email: 24f2009374@ds.study.iitm.ac.in

Roll Number: 24f2009374

Enrollment: Diploma in Programming

Project Details

The project has been initialized with all core functionalities along with some recommended and extra functionalities as needed to give a seamless experience. JavaScript, RESTful API, Vue handling, Bootstrap, Axios and ChartJS have been used extensively in this project.

The project base has been initialized with a similar approach as V1, via the “factory-app” pattern wherein the main entry point of the app calls the app-creation method from the `__init__.py` and is handled skillfully across the project to avoid any circular or illegal imports. Blueprint routes have been used due to the extensive approach in sending the static HTML files. SQLite has been used as the main database setup and have been setup using Flask-SQLAlchemy.

Using the experience from the previous project, I approached this project with a hands-on and optimal setup plans in mind. The first things created after the initial backend setup were the base templates and a proper user / admin separation of concerns regarding related files to avoid confusion. Similar to the previous term, I focused on the Admin side first to initialize all major CRUD operations and other such functionalities and build from there.

At the same time, a detailed Flask-Security based login system was also created and validated using front-end JS.

To avoid time-dumping into styling, a very similar style of design has been inherited from the previous template but has been polished and expanded upon with a higher degree of polish and finesse.

All CRUD resources were initialized with detailed and approachable API endpoints for everything and the routing pattern was modularized along with the JSON being sent over to avoid cluttering of routing-links and also act as a base for ease of navigation from places like search.

The subjects came first and was easily implemented with Vue and Axios, which were used from the get-go to avoid migration issues later on. The chapters and Quizzes in similar fashion and then came the Questions. I've implemented both front-end JS based sanity checks and validations along with backend checks for deeper-level validations. The Questions, having an elegant and modular method of being implemented, being able to have 2-4 options per question and has been made effective with sufficient checks to avoid complications later on. After this, PUT and DELETE operations were added and tested.

Once these were in place, it was onto the User side. To avoid a lot of complication and navigation issues for both admin and user, a lot of functionality has been levied to the navbar and this is no different. The enrollment functionality stays the same and the attempt-counter is also properly managed along with sanity checks to avoid Axios errors for empty quizzes. At the same time, a View Score was added to record and view all user quiz attempts for that quiz.

The Quiz itself was quite complicated, first came the rendering itself after which came the JS functions required to make it work like one. Once the main submission setup, checking, record and timestamping was completed, I added a fully functional JS Timer that would Force Submit the quiz if it ran out and redirect the user to a thank you page.

After this, I added the Search functionality for Admin and User. The Admin can search for anything and go *into* those specific parts of the Database and edit them as required. At the same time, the User can only search the quizzes they wish to enroll in.

After this, I added a lot of charts using ChartJS which is dynamic, unlike matplotlib which is static and requires saving, taking up extra space and in my experience, even broke during the project documentation. And even disregarding the instability, the very aesthetic of ChartJS matched my project very well. I added summary charts for both User and Admin as per requirements.

After this, I added Redis caching, which had some errors so I had to use json request viewing within the cache prefix to make each part have an individual cache (users, chapters, subjects, etc) to avoid overriding displays from using the same APIs per display. After this, I added a basic Mail setup using MailHog to act as a fake SMPT server and using Flask-Mail to create and send mail.

After this, I added the celery backjobs, both automatic and manually triggered upon certain conditionals such as new quiz created and quiz submission. I added

automatic ones for daily Quiz reminders and monthly summarizations using html as well which are send by mail over to MailHog. The tasks had to be maintained within the same file as Celery since the factory app pattern was conflicting and resulting in a circular import, breaking celery itself. In similar fashion, CSV exports were added for User-Quiz summary which can be extracted manually from the User Dashboard.

Project Functionalities

Super-User (Admin) and User Distinction

HTML Serving using Blueprint Routing and VueJS to handle rendering and functions

Well-made and simple Schemas

Multiple API Endpoints for different requirements.

Security and Login using JS validations and Flask-Security

CRUD for all application parts for Admin Control over App.

Customizable SCQ Question Creation

JavaScript based Validations.

Timer within Quiz with Force Submit.

Previous Attempts Viewable

Detailed Views for all Components.

Detailed Search for User and Admin with working Links.

Enrollment function for User.

Summary Charts for User and Admin using ChartJS

Dynamic Redis-Caching for API and App Performance

Mail Setup for MailHog Fake SMPT Testing

Celery Backjobs to send mails, both automatic (daily+monthly) and conditional(quiz creation+quiz submission)

Polished Bootstrap and CSS based styling

Frameworks and Libraries used (via “pip freeze” Extraneous or Unused libraries not listed)

async-timeout==5.0.1

cachelib==0.13.0

celery==5.5.3

Flask==3.1.1

Flask-Caching==2.3.1

Flask-Login==0.6.3

Flask-Mail==0.10.0

Flask-RESTful==0.3.10

Flask-SQLAlchemy==3.1.1

kombu==5.5.4

python-dateutil==2.9.0.post0

pytz==2025.2

redis==6.2.0

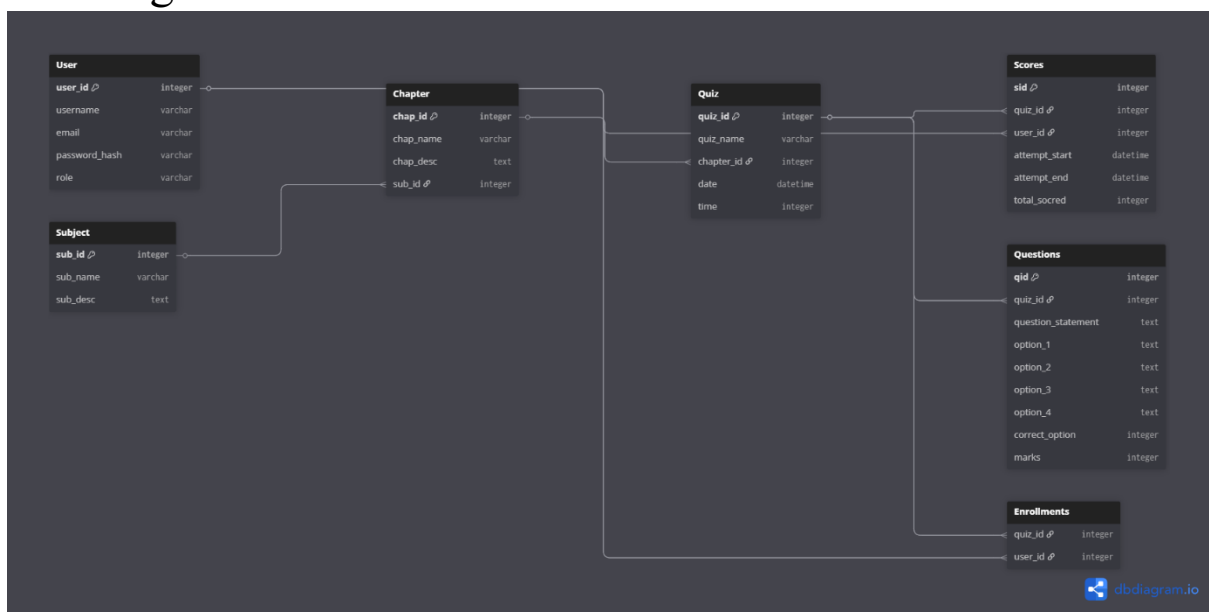
SQLAlchemy==2.0.41

typing_extensions==4.14.1

wcwidth==0.2.13

Werkzeug==3.1.3

ER Diagram



Drive Link for Video:

https://drive.google.com/file/d/1JP4X_rl7gFx8we0CbOL8doqBflFJh6Fd/view?usp=drive_link